ARTIFICIAL INTELLIGENCE

Integrated Experiments — CO4, Assessment Tool 3 (AT-3)

CO4: Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)

Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on the Iris dataset using the scikit-learn library, covering data pre-processing, model construction using the Gini Index, and full performance evaluation.

(a) Data Loading, Pre-processing, and Train-Test Split

The Iris dataset (150 samples, 3 classes: setosa, versicolor, virginica; 4 numeric features) was loaded using sklearn.datasets.load_iris(). The dataset was checked for missing values and the target class was label-encoded (0, 1, 2), which is already numeric so no additional encoding was required. The data was split 70:30 into training and testing sets using stratified sampling to preserve class balance.

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['species'] = iris.target
df['species_name'] = df['species'].map(dict(enumerate(iris.target_names)))

print(df.head())
print(df.dtypes)
print(df.isnull().sum())          # check missing values

X = df[iris.feature_names]
y = df['species']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)
```

Output — First 5 rows

sepal length	sepal width	petal length	petal width	species	species_name
5.1	3.5	1.4	0.2	0	setosa
4.9	3.0	1.4	0.2	0	setosa
4.7	3.2	1.3	0.2	0	setosa
4.6	3.1	1.5	0.2	0	setosa
5.0	3.6	1.4	0.2	0	setosa

Output — Data types & missing values

```
sepal length (cm)    float64
sepal width (cm)     float64
petal length (cm)    float64
petal width (cm)     float64
species               int64
species_name          object (str)
```

```
Missing values in every column: 0   (Iris is a clean, complete dataset)

Training set size: 105 rows
Testing set size : 45 rows
```

(b) Building the Decision Tree (Gini Index)

A DecisionTreeClassifier was trained using criterion='gini' (the default CART splitting criterion), which measures node impurity. At each node, the algorithm selects the feature and threshold that produce the greatest reduction in Gini impurity between parent and child nodes.

```
from sklearn.tree import DecisionTreeClassifier, export_text

clf = DecisionTreeClassifier(criterion='gini', max_depth=4, random_state=42)
clf.fit(X_train, y_train)

print(export_text(clf, feature_names=list(iris.feature_names)))

root_feature = iris.feature_names[clf.tree_.feature[0]]
root_threshold = clf.tree_.threshold[0]
print(f"Root splits on {root_feature} <= {root_threshold:.2f}")
```

Output — Tree structure (text)

```
|--- petal length (cm) <= 2.45
|   |--- class: 0  (setosa)
|--- petal length (cm) >  2.45
|   |--- petal width (cm) <= 1.55
|   |   |--- petal length (cm) <= 4.95
|   |   |   |--- class: 1  (versicolor)
|   |   |--- petal length (cm) >  4.95
|   |   |   |--- class: 2  (virginica)
|   |--- petal width (cm) >  1.55
|   |   |--- petal width (cm) <= 1.70
|   |   |   |--- sepal width (cm) <= 2.85
|   |   |   |   |--- class: 1  (versicolor)
|   |   |   |--- sepal width (cm) >  2.85
|   |   |   |   |--- class: 2  (virginica)
|   |   |--- petal width (cm) >  1.70
|   |   |   |--- petal length (cm) <= 4.85
|   |   |   |   |--- class: 1  (versicolor)
|   |   |   |--- petal length (cm) >  4.85
|   |   |   |   |--- class: 2  (virginica)
```

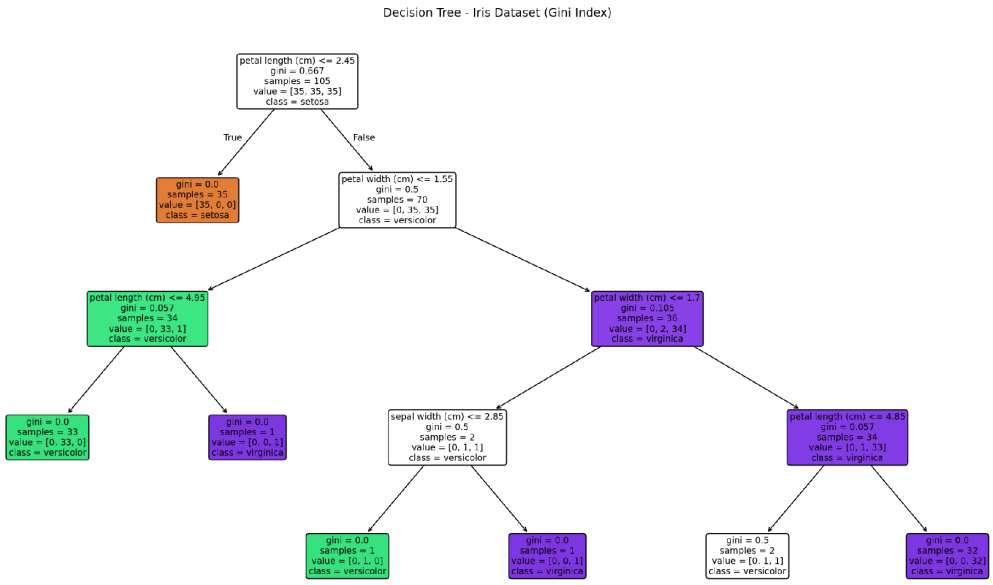

Fig 1.1 — Visualised Decision Tree for the Iris dataset (Gini index)

Root Node Identification & Justification

Root node split: petal length (cm) <= 2.45

Gini impurity at root: 0.667 (maximum impurity, since the training set contains an equal 35/35/35 mix of the three classes)

Feature importances computed from the total Gini-impurity reduction each feature contributes:

Feature	Importance (Gini-based)
petal length (cm)	0.549
petal width (cm)	0.436
sepal width (cm)	0.014
sepal length (cm)	0.000

Justification: petal length is chosen as the root because it produces the single purest split in the entire training set — every sample with petal length ≤ 2.45 cm is a setosa (Gini = 0 in that child node), giving the largest possible impurity reduction of any first split. This matches the highest feature-importance score (0.549) among all four attributes, confirming petal length as the most discriminative feature and hence the natural choice for the root node.

(c) Model Evaluation

```
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

y_pred = clf.predict(X_test)
print('Accuracy:', accuracy_score(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred, target_names=iris.target_names))
```

Output — Accuracy

```
Accuracy Score: 0.8889  (88.89%)
```

Output — Confusion Matrix

Actual \ Predicted	setosa	versicolor	virginica
setosa	15	0	0
versicolor	0	12	3
virginica	0	2	13

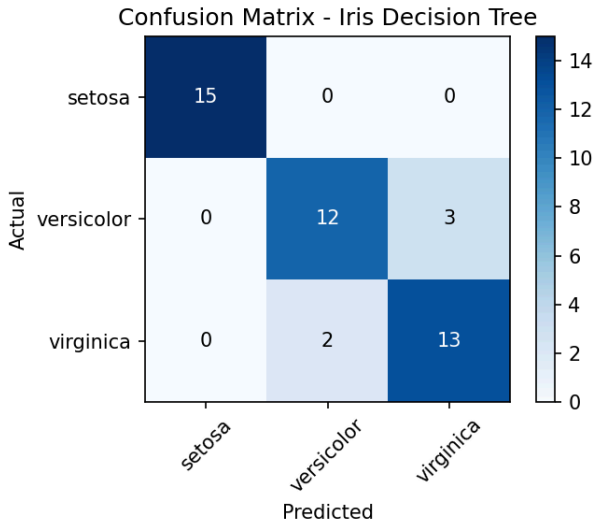

Fig 1.2 — Confusion matrix heatmap

Output — Classification Report

Class	Precision	Recall	F1-score	Support
setosa	1.00	1.00	1.00	15
versicolor	0.86	0.80	0.83	15
virginica	0.81	0.87	0.84	15
Accuracy			0.89	45
Macro avg	0.89	0.89	0.89	45
Weighted avg	0.89	0.89	0.89	45

Performance Comment

The classifier achieves 88.89% accuracy on unseen test data. Setosa is classified with perfect precision and recall (1.00), since it is linearly separable from the other two species using petal length alone. All errors occur between versicolor and virginica, which is expected because these two species overlap in petal width/length near the 1.55–1.70 cm boundary region. Virginica has a slightly higher recall (0.87) than versicolor (0.80), meaning the model is a little more likely to mistake a versicolor flower for virginica than the reverse.

Misclassified Instances (at least two, from the 5 found)

Test Row Index	Sepal L	Sepal W	Petal L	Petal W	Actual	Predicted
56	6.3	3.3	4.7	1.6	versicolor	virginica
138	6.0	3.0	4.8	1.8	virginica	versicolor
85	6.0	3.4	4.5	1.6	versicolor	virginica

Test Row Index	Sepal L	Sepal W	Petal L	Petal W	Actual	Predicted
77	6.7	3.0	5.0	1.7	versicolor	virginica
106	4.9	2.5	4.5	1.7	virginica	versicolor

Rows 56 and 85 are true versicolor flowers with a petal width of 1.6 cm — just above the 1.55 cm split threshold used inside the tree — causing the model to route them into the virginica branch. This confirms that most misclassifications occur near the decision boundary the tree has learned, which is typical behaviour for a Decision Tree on overlapping classes.

Experiment 2: Reinforcement Learning – Q-Learning

Objective: Implement tabular Q-Learning for a 4×4 Grid World environment, train the agent for 100 episodes, and analyse how the Q-table and cumulative reward evolve as the agent learns an optimal policy.

(a) Environment Definition

The environment is a 4x4 grid of 16 states, numbered 0–15 in row-major order (state = row*4 + col). The agent starts at state 0 (top-left) and must reach the goal at state 15 (bottom-right).

Grid layout (state numbers)			
0	1	2	3
4	5 (Obstacle)	6	7 (Obstacle)
8	9	10	11 (Obstacle)
12	13	14	15 (Goal)

States: 16 (4x4 grid, indexed 0–15)

Actions: Up, Down, Left, Right (4 actions per state)

Reward structure: +10 on reaching the Goal state (15); -1 for every normal step; -5 on entering an obstacle cell (states 5, 7, 11)

Q-table initialisation: np.zeros((16, 4)) — all Q-values start at 0

Hyperparameters: learning rate α = 0.1, discount factor γ = 0.9, exploration rate ε = 0.2 (ε-greedy policy)

```
import numpy as np

GRID_SIZE = 4
N_STATES, N_ACTIONS = 16, 4
ACTIONS = ['Up', 'Down', 'Left', 'Right']
GOAL_STATE = 15
OBSTACLE_STATES = [5, 7, 11]

def step(state, action):
    if state == GOAL_STATE: return state, 0, True
    r, c = divmod(state, GRID_SIZE)
    if action == 'Up':    r = max(r-1, 0)
    if action == 'Down':  r = min(r+1, GRID_SIZE-1)
    if action == 'Left':  c = max(c-1, 0)
    if action == 'Right': c = min(c+1, GRID_SIZE-1)
    ns = r*GRID_SIZE + c
    if ns == GOAL_STATE:        return ns, 10, True
    elif ns in OBSTACLE_STATES: return ns, -5, False
    else:                       return ns, -1, False

Q = np.zeros((N_STATES, N_ACTIONS))     # initial Q-table
alpha, gamma, epsilon = 0.1, 0.9, 0.2   # hyperparameters
```

(b) Running Q-Learning for 100 Episodes

The agent was trained for 100 episodes (max 50 steps/episode) starting from state 0 each time, using an epsilon-greedy behaviour policy and the standard Q-Learning update rule:

```
Q(s,a)  <-  Q(s,a) + alpha * [ r + gamma * max_a' Q(s',a')  -  Q(s,a) ]
```

```
for episode in range(1, 101):
    state = 0
    for t in range(50):
        if np.random.rand() < epsilon:
            a = np.random.randint(N_ACTIONS)          # explore
        else:
            a = np.argmax(Q[state])                    # exploit
        next_state, reward, done = step(state, ACTIONS[a])
        best_next = np.max(Q[next_state])
        Q[state, a] += alpha * (reward + gamma*best_next - Q[state, a])
        state = next_state
        if done: break
```

Q-value Evolution — Tracked States 0 (start) and 10 (mid-grid)

Episode	State	Up	Down	Left	Right
1	0	-0.199	-0.199	-0.271	-0.100
1	10	-0.100	0.000	0.000	0.000
50	0	-2.279	-2.047	-2.249	-2.163
50	10	-0.303	7.462	0.096	-0.986
100	0	-2.071	1.168	-2.308	-2.240
100	10	0.506	7.992	1.825	-0.821

At Episode 1 almost all Q-values are close to zero since the agent has barely explored the environment. By Episode 50, state 10 already shows a strong preference for the 'Down' action (Q ≈ 7.46) — the direction that leads most directly toward the goal at state 15 — while state 0's values are still negative across all actions because it is far from the goal and every path there accumulates step penalties. By Episode 100 the values have largely stabilised: state 10's 'Down' Q-value has increased slightly further to 7.99 (approaching the theoretical value) and state 0 now shows 'Down' as clearly the best action (1.168), showing the value information has propagated backward from the goal toward the start state.

Final Q-table (rounded to 2 decimals, rows = states 0–15, columns = Up/Down/Left/Right)

```
State  0: [-2.07,  1.17, -2.31, -2.24]
State  1: [-1.79, -2.75, -1.70, -1.51]
State  2: [-1.12,  0.30, -1.32, -1.04]
State  3: [-1.13, -1.38, -1.08, -1.14]
State  4: [-1.55,  2.76, -0.89, -3.52]
State  5: [-0.65,  1.49, -0.64, -0.47]   (obstacle)
State  6: [-0.66,  4.30, -1.38, -1.75]
State  7: [-0.52, -0.95, -0.24, -0.50]   (obstacle)
State  8: [-1.13, -0.85, -0.88,  4.44]
State  9: [-2.07,  0.15, -0.05,  6.16]
State 10: [ 0.51,  7.99,  1.83, -0.82]
State 11: [-0.50,  6.13,  0.00, -0.08]   (obstacle)
State 12: [-0.53, -0.65, -0.49,  0.24]
State 13: [-0.21, -0.20, -0.41,  4.54]
State 14: [ 2.29,  4.75,  0.12, 10.00]
State 15: [ 0.00,  0.00,  0.00,  0.00]   (goal - terminal)
```

(c) Final Learned Policy & Convergence

```
policy = []
for s in range(N_STATES):
    policy.append('G' if s==GOAL_STATE else ('X' if s in OBSTACLE_STATES
                  else '^v<>'['UDLR'.index('UDLR'[np.argmax(Q[s])])]))
print(np.array(policy).reshape(4,4))
```

Output — Learned policy grid

Down	Right	Down	Left
Down	Obstacle	Down	Obstacle
Right	Right	Down	Obstacle
Right	Right	Right	GOAL

Reading the grid row by row, the arrows form a continuous path that always steers the agent around the three obstacle cells (5, 7, 11) and down/right toward the goal at state 15 — e.g. from state 0: Down -> 4 -> Down -> 8 -> Right -> 9 -> Right -> 10 -> Down -> 14 -> Right -> 15 (Goal). This is a valid shortest safe path given the obstacle layout.

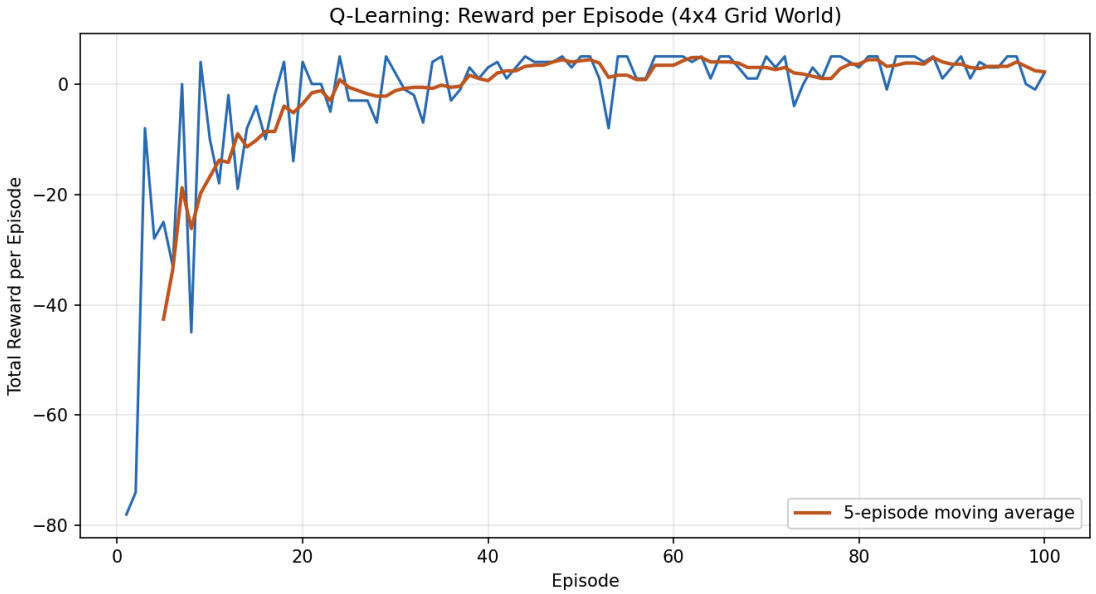

Fig 2.1 — Cumulative reward per episode, with 5-episode moving average

Convergence Behaviour — Justification

- Early episodes (1–20): rewards are strongly negative (e.g. -78, -74) because the ε-greedy agent is mostly taking random/exploratory actions with almost no knowledge of the environment, frequently wandering into obstacles or taking long routes.
- Middle episodes (20–60): rewards rise sharply and become noisy/oscillating as the Q-table starts encoding useful state-action values, but the ε = 0.2 exploration rate still occasionally forces sub-optimal detours.
- Late episodes (60–100): the moving-average reward stabilises in a small positive band (around 0 to +5 per episode), matching the near-optimal path length (≈5-6 steps at -1 each) plus the +10 goal bonus, confirming the policy has converged close to the optimal solution.

The remaining variance even at Episode 100 is expected and does not indicate failure to converge — it is caused by the fixed ε = 0.2 exploration rate, which continues to force occasional random (sometimes costly) actions by design so the agent keeps sampling the environment. Decaying ε over time would further reduce this variance and tighten convergence.